

RXJS

- „Reactive Extension Library for JavaScript“
- Fokus auf Asynchronität, reaktive Programmierung
- Im Gegensatz zu Promises werden wiederholt Werte erzeugt
- Viele Operator-Funktionen um Anpassungen vornehmen zu können

GRUNDBAUSTEINE

- Observable
 - Sammlung zukünftiger Werte / Ereignisse

```
class RxJSExample {  
  // example ist das sogenannte Observable  
  readonly example$ = new Observable((subscriber) => {  
    subscriber.next('Value A');  
    subscriber.next('Value B');  
    setTimeout(() => {  
      subscriber.next('Value C');  
      subscriber.complete();  
    }, 500);  
  });  
}
```

- Observer
 - Konsument der empfangenen Werte

```
class RxJSExample {
  readonly example$ =; // ...

  constructor() {
    const subscription = this.example$
      .pipe(map((value) => 'Tolles ' + value))
      .subscribe(
        // dieser Callback ist der Observer
        (v) => {
          console.log(v)
        });
  }
}
```

- Subscription
 - Entspricht der Ausführung des Observables

```
class RxJSExample {
  readonly example$ =; // ...

  constructor() {
    // der Rückgabewert aus .subscribe ist die Subscription
    const subscription = this.example$
      .pipe(map((value) => 'Tolles ' + value))
      .subscribe(
        (v) => {
          console.log(v)
        });
  }
}
```

- Operator
 - Ermöglicht das Anpassen
 - der Werte
 - des Observables (Stichwort: Higher Order Observables)

```
class RxJSExample {
  readonly example$ =; // ...

  constructor() {
    const subscription = this.example$
      .pipe(
        // map ist ein Operator der den durchgereichten Wert verändert
        map((value) => 'Tolles ' + value))
      .subscribe(
        (v) => {
          console.log(v)
        });
  }
}
```

- Subject
 - Ermöglicht das multicasten von Werten an diverse Observer

```
class RxJSExample {  
  readonly subject$ = new Subject();  
  
  constructor() {  
    const subscription = this.subject$  
      .pipe(map((value) => 'Tolles ' + value))  
      .subscribe((v) => console.log(v));  
    subject$.next('Value');  
  }  
}
```

SUBJECTS

- Subject
 - Werte die nach der Subscription entstehen werden an den Observer weitergereicht
- ReplaySubject
 - Ähnlich wie Subject, allerdings kann ein Puffer festgelegt werden
 - Werte die noch im Puffer liegen, werden an den Observer weitergegeben
- BehaviourSubject
 - Enthält einen aktuellen Wert, welcher bei Subscription direkt an den Observer gereicht wird

SUBSCRIPTION LIFECYCLE

- Erstellung

```
const obsv$ = new Observable();
```

- Subscription

```
const obsv$ = new Observable();  
obsv$.subscribe();
```

- Ausführung

```
const obsv$ = new Observable((subscriber) => {  
  subscriber.next('Value');  
});  
obsv$.subscribe();
```

- Finalisierung

```
const obsv$ = new Observable((subscriber) => {  
  subscriber.next('Value');  
});  
const sub = obsv$.subscribe();  
sub.unsubscribe();
```

oder

```
const obsv$ = new Observable((subscriber) => {  
  subscriber.next('Value');  
  subscriber.complete();  
});  
obsv$.subscribe();
```

- Handhabung des Subscription Lifecycle wichtig
 - Subscriptions müssen Zustand "complete" oder "error" erreichen
 - Sonst Gefahr: Memory Leaks und unerwartete Seiteneffekte, wenn Subscriptions unerwartet weiter laufen

MÖGLICHKEITEN

- Filter-Operatoren wie "take", "first"

```
this.example$.pipe(take(1)).subscribe();  
this.example$.pipe(first()).subscribe();
```

- Expliziter "unsubscribe"-Aufruf an der Subscription

```
class ExampleComponent {  
  readonly sub!: Subscription;  
  
  constructor() {  
    this.sub = this.example$.subscribe();  
  }  
  
  ngOnDestroy() {  
    this.sub.unsubscribe();  
  }  
}
```

- Verwenden eines Subjects in Kombination mit "takeUntil"-Operator

```
class ExampleComponent {
  readonly destroy$ = new Subject();

  constructor() {
    this.example$.pipe(takeUntil(this.destroy$)).subscribe();
  }

  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

- Nutzen von "DestroyRef" + "takeUntilDestroyed"-Operator
 - Verfügbar in Komponenten, Direktiven, Services

```
class ExampleComponent {
  readonly destroyRef = inject(DestroyRef);

  constructor() {
    this.example$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe();
  }
}
```

- Nutzen der "async"-Pipe von Angular

```
class ExampleComponent {
  readonly cities$ = this.cityService.getAllCities();
}
```

```
<city-list [cities]="cities$ | async"/>
```

HOT VS COLD

- Cold Observables
 - Werte entstehen nur, wenn eine Subscription besteht
 - Die Data-Source entsteht beim Aufruf der Subscribe-Methode
 - Cold Observables sind somit unicasts
- Hot Observables
 - Werte entstehen zu beliebiger Zeit, auch wenn keine Subscription besteht
 - Die Data-Source entsteht außerhalb von Subscribe-Aufrufen
 - Hot Observables sind somit multicasts

- Beispiel Cold Observables
 - Analogie: Klasse und Klasseninstanzen
 - Subscribe erzeugt die eigentliche Data-Source erst (hier: Math.random())

```
const coldObsv$ = of(null).pipe(map(() => Math.random()));
coldObsv$.subscribe(console.log);
coldObsv$.subscribe(console.log);
coldObsv$.subscribe(console.log);
```

- Beispiel Hot Observables
 - Data-Source entsteht außerhalb der Subscription

```
const random = Math.random();
const coldObsv$ = of(null).pipe(map(() => random));
coldObsv$.subscribe(console.log);
coldObsv$.subscribe(console.log);
coldObsv$.subscribe(console.log);
```

- Beispiele
 - Hot Observables
 - fromEvent
 - Subject / BehaviourSubject / ReplaySubject
 - share / shareReplay / etc.
 - Cold Observables
 - HttpClient.get / HttpClient.post / etc.

OPERATOREN (AUSZUG)

- Creation
 - fromEvent
 - Of

- JoinCreation
 - combineLatest
 - merge
 - zip
 - race

- Transformation
 - mergeMap
 - switchMap
 - map

- Filtering
 - filter
 - skip
 - take
 - distinctUntilChanged

- Join
 - startWith
 - withLatestFrom / concatLatestFrom

- Multicasting
 - shareReplay

- ErrorHandling
 - retry
 - catchError

BEST PRACTISES

- Benennt Observable-Attribute immer mit einem \$-Suffix (z.B. users\$ statt users)
 - Sofort erkennbar dass das ein asynchroner Wert ist
- Verschachtelt niemals subscribe() Aufrufe
 - Es entsteht ähnlich zu Callbacks eine „Pyramid of Doom“
 - Stattdessen: Operatoren wie mergeMap(), switchMap()
- Nutzt so häufig wie möglich die async-Pipe von Angular
 - Sie handelt den Lifecycle von sich aus und die Child-Component kann einfach mit den direkten Werten arbeiten
- Macht euch mit den Operatoren vertraut
 - <https://www.learnrxjs.io/learn-rxjs/operators>

INTERCEPTOREN

- Classes die `HttpInterceptor`-Interface implementieren
- Müssen im entsprechenden Module via `InjectionToken` provided werden
- Fangen ausgehende Requests ab und modifizieren sie
- Beliebige viele Interceptoren können aktiviert sein
- Nützlich für z.B. Anhängen eines Bearer-Tokens an Requests (Authentifizierung)

• Beispiel

```
@Injectable()
export class TokenInterceptor implements HttpInterceptor {
  readonly tokenService = inject(TokenService);

  // intercept wird bei jedem ausgehenden Request aufgerufen
  intercept(
    req: HttpRequest<unknown>,
    next: HttpHandler
  ): Observable<HttpEvent<unknown>> {
    return this.tokenService.bearerToken$.pipe(
      mergeMap((token: string) => {
        // Modifizierung des aktuellen Requests
        const headers = req.headers.append('Authorization', 'Bearer ' + token);
        const request = req.clone({headers});
        // Weitergabe des angepassten Requests an den nächsten Interceptoren
        return next.handle(request);
      })
    );
  }
}

// Provider für diesen Interceptor
export const tokenInterceptorProviders = [
  {provide: HTTP_INTERCEPTORS, useClass: TokenInterceptor, multi: true},
];
```